

Módulo 4 – Capítulo 04 – Sección 01

Arquitecturas Basadas en Agentes

Un sistema RAG, por sofisticado que sea, es fundamentalmente reactivo: recibe una consulta, recupera contexto y genera una respuesta. No puede iniciar acciones, encadenar múltiples pasos de razonamiento con herramientas externas, o adaptar su estrategia cuando el primer intento no produce el resultado esperado. Para los casos de uso que requieren esas capacidades — automatizar un proceso de múltiples pasos, investigar un problema iterando sobre herramientas, ejecutar una transacción en un sistema externo basándose en la interpretación de lenguaje natural — el patrón RAG no es suficiente. Se necesita una arquitectura de agentes.

El concepto de agente, en el contexto de sistemas de IA basados en LLMs, tiene una definición arquitectónica precisa: un agente es un sistema que utiliza un modelo de lenguaje como motor de razonamiento para decidir qué acciones tomar, ejecuta esas acciones mediante herramientas, observa los resultados de esas acciones, y utiliza esas observaciones para decidir el siguiente paso, hasta alcanzar un objetivo o agotarse por un límite configurado. Esta definición distingue a un agente de un chatbot sofisticado o de un pipeline RAG: el agente tiene autonomía para decidir el camino hacia el objetivo, no solo para generar texto.

El lector familiarizado con los patrones Planning y Delegation del Módulo 3 reconocerá los fundamentos conceptuales de las arquitecturas de agentes. El patrón Planning describe cómo instruir a un LLM para que descomponga un objetivo complejo en pasos ejecutables. El patrón Delegation describe cómo un agente orquestador puede asignar subtarefas a sub-agentes especializados. Las arquitecturas de agentes del Módulo 4 toman esos patrones y los convierten en sistemas productivos: con estado persistente, herramientas reales, mecanismos de fallo controlado, observabilidad y límites de autonomía.

Los temas que este capítulo desarrolla son:

- **Componentes de un agente:** el modelo de razonamiento, los tipos de memoria, el registro de herramientas, y el bucle de observación-acción que define el comportamiento del agente.

- **Memoria y estado:** cómo un agente mantiene contexto entre interacciones, cuándo usar memoria episódica versus memoria semántica, y cómo gestionar el estado en agentes de larga duración.
- **Patrones de diseño:** los patrones ReAct, Planner-Executor, Supervisor-Workers y Reflection, con criterios para elegir entre ellos y sus fallos característicos.
- **Casos de uso empresariales:** los escenarios donde los agentes producen valor real y los escenarios donde añaden complejidad sin beneficio.

Nota del Arquitecto: *La principal trampa de las arquitecturas de agentes es pensar que "más autonomía es mejor". En la práctica, cada grado de autonomía que se concede a un agente aumenta el espacio de comportamientos posibles, incluidos los no deseados. Un agente que puede ejecutar código, leer y escribir archivos, y hacer llamadas a APIs externas tiene una superficie de impacto enorme si su razonamiento se desvía del objetivo. Los mejores sistemas agénticos que he visto en producción son los que definen con precisión qué puede hacer el agente, qué no puede hacer, y en qué condiciones debe pausar y solicitar confirmación humana antes de continuar.*

El valor real de las arquitecturas de agentes no reside en el modelo de lenguaje que las impulsa, sino en la cuidadosa ingeniería de sus componentes, la robustez de sus herramientas y la precisión de sus límites de autonomía. Las secciones siguientes desarrollan cada uno de estos elementos.

Módulo 4 – Capítulo 04 – Sección 02

Componentes de un Agente

Un agente de IA productivo no es un LLM con acceso a herramientas. Es un sistema con arquitectura interna propia, donde cada componente tiene una responsabilidad específica y una interfaz definida. Entender esa arquitectura interna es el requisito previo para diseñar agentes que sean predecibles, mantenibles y observables. La ausencia de ese diseño explícito es la causa más frecuente de agentes que funcionan en demos pero fallan en producción.

El primer componente es el modelo de razonamiento, que es el motor de toma de decisiones del agente. El modelo recibe el contexto actual — el objetivo, el historial de pasos previos, las observaciones de las herramientas ejecutadas — y produce la decisión del siguiente paso: qué herramienta invocar, con qué parámetros, o si el objetivo ha sido alcanzado. La elección del modelo de razonamiento es una decisión arquitectónica con consecuencias significativas en costo y latencia: modelos más potentes (GPT-4o, Claude 3.5 Sonnet) producen mejor razonamiento en tareas complejas pero son más costosos por llamada. En un agente que puede ejecutar decenas de pasos para completar una tarea, el costo acumulado de tokens puede ser sustancial. Una estrategia es utilizar un modelo ligero para pasos de razonamiento simples y un modelo potente solo cuando se detecta alta complejidad.

El segundo componente es la memoria, y es donde la arquitectura de agentes se diferencia de manera más significativa de un chatbot. La memoria de un agente opera en dos niveles fundamentales:

- **Memoria episódica (corto plazo):** el contexto de la sesión actual o tarea en curso. Incluye el objetivo, los pasos ejecutados, las observaciones recibidas y el estado actual. Esta memoria vive en la ventana de contexto del modelo o en una estructura de datos en memoria que se serializa y deserializa entre llamadas al LLM. Su limitación es la ventana de contexto: a medida que el agente ejecuta más pasos, el historial crece y puede superar el límite del modelo.
- **Memoria semántica (largo plazo):** conocimiento persistente entre sesiones. Puede incluir el perfil del usuario, las preferencias aprendidas, el resultado de investigaciones previas, o el estado de tareas de larga duración. Esta memoria se almacena externamente

— en una base de datos relacional, un vector store o un sistema de almacenamiento de clave-valor — y el agente la consulta cuando es relevante para la tarea actual.

La gestión del crecimiento de la memoria episódica es un problema de diseño real. Estrategias como la compresión del historial (resumir pasos previos en lugar de conservarlos íntegros), la paginación del contexto, o el archivado selectivo de pasos relevantes deben considerarse desde el diseño inicial para agentes destinados a tareas de larga duración.

El tercer componente es el registro de herramientas (tool registry). Las herramientas son las acciones que el agente puede ejecutar: búsquedas en bases de datos, llamadas a APIs externas, ejecución de código, lectura y escritura de archivos, o cualquier función del sistema que haya sido envuelta con una interfaz compatible con el protocolo de herramientas del modelo. El registro de herramientas es el catálogo de capacidades del agente: qué puede hacer, cómo se llama cada herramienta, qué parámetros acepta y qué devuelve. La calidad de las descripciones de las herramientas en el registro es crítica: el modelo decide qué herramienta usar basándose en esas descripciones, y una descripción ambigua produce invocaciones incorrectas.

El cuarto componente es el bucle de observación-acción (observation-action loop), que es el ciclo de ejecución del agente. El bucle tiene la siguiente estructura: el modelo de razonamiento recibe el estado actual y produce una acción (herramienta a invocar y parámetros); la herramienta se ejecuta y produce una observación (resultado de la ejecución); la observación se incorpora al contexto; y el ciclo se repite hasta que el modelo decide que el objetivo ha sido alcanzado o se supera un límite configurado de iteraciones. La gestión de fallos en el bucle — qué sucede si la herramienta devuelve un error, si el modelo produce una invocación con formato incorrecto, o si el timeout del paso es superado — debe estar explícitamente diseñada.

Los parámetros de configuración del agente que deben establecerse en el diseño incluyen:

- **Número máximo de iteraciones:** límite absoluto del número de pasos antes de terminar con error. Previene bucles infinitos.
- **Timeout por paso:** tiempo máximo permitido para la ejecución de una herramienta individual.
- **Política de reintento:** qué hacer si una herramienta falla (reintentar, usar herramienta alternativa, escalar a humano).
- **Umbral de confianza:** cuándo el agente debe pausar y solicitar confirmación antes de ejecutar una acción de alto impacto.

La arquitectura interna de un agente bien diseñado convierte al modelo de lenguaje en un tomador de decisiones informado, no en un generador de texto sin control. La siguiente sección explora uno de los aspectos más complejos de los agentes de larga duración: cómo gestionar el estado y la memoria cuando la tarea se extiende a través de múltiples sesiones o cuando el contexto crece más rápido de lo que la ventana del modelo puede absorber.

Módulo 4 – Capítulo 04 – Sección 03

Memoria y Estado del Agente

La diferencia entre un agente que completa tareas simples y un agente capaz de gestionar procesos de negocio complejos que se extienden en el tiempo es su capacidad de mantener estado. Un agente sin gestión de estado es, en esencia, un pipeline: recibe una entrada, ejecuta una secuencia de pasos y produce una salida. Un agente con gestión de estado puede retomar una tarea interrumpida, adaptar su comportamiento basándose en interacciones previas, y acumular conocimiento sobre su dominio de operación entre sesiones. Diseñar la capa de estado es una de las decisiones arquitectónicas más importantes y menos discutidas en las arquitecturas de agentes.

El problema central de la gestión de estado en agentes LLM es la tensión entre la naturaleza sin estado de los modelos de lenguaje y la necesidad de continuidad en el tiempo. Cada llamada al LLM es independiente: el modelo no "recuerda" conversaciones previas a menos que esa información se incluya explícitamente en el prompt. Esto significa que todo el estado que el agente necesita debe ser serializado, almacenado externamente y deserializado en cada iteración del bucle de razonamiento.

El estado de un agente productivo se organiza en tres capas con diferentes características de persistencia y acceso:

Estado de sesión (efímero, duración de una tarea): incluye el objetivo de la tarea actual, el plan inicial si existe, el historial de acciones y observaciones de la sesión, y cualquier variable de trabajo creada durante la ejecución. Este estado vive típicamente en memoria durante la ejecución y se serializa a un almacenamiento persistente solo si la tarea puede interrumpirse y reanudarse. Redis o una base de datos clave-valor con TTL corto son opciones naturales para este nivel. Cuando el historial de sesión crece y amenaza con superar la ventana de contexto del modelo, la estrategia de compresión progresiva — resumir bloques del historial más antiguo mientras se mantiene el detalle del historial reciente — permite gestionar la longitud sin perder el hilo de la tarea.

Estado de usuario o entidad (persistente, duración indefinida): incluye el perfil del usuario, sus preferencias, el historial de interacciones relevantes anteriores, y cualquier información que el agente deba recordar entre sesiones distintas. Por ejemplo, un agente de soporte que ha resuelto un problema técnico específico para un usuario debe recordar esa solución si el mismo problema reaparece semanas después. Este estado se almacena en una base de datos relacional o documental y se recupera al inicio de cada sesión para el usuario correspondiente. La gestión de privacidad de este estado — qué se retiene, por cuánto tiempo, quién puede acceder — es un requisito no funcional que debe definirse antes del despliegue.

Memoria semántica (persistente, acceso por similitud): cuando el volumen de conocimiento acumulado por el agente supera lo que puede incluirse en el contexto de cada llamada, se usa una base vectorial como memoria semántica. El agente puede escribir en esta memoria ("aprendí que el cliente X prefiere respuestas breves con ejemplos de código") y recuperar información relevante al inicio de cada sesión o cuando el razonamiento lo requiere. Esta arquitectura permite que el agente "aprenda" sin reentrenamiento del modelo subyacente.

La gestión del estado en agentes de larga duración requiere resolver dos problemas adicionales. El primero es la coherencia: cuando una tarea es interrumpida (por un timeout, un error o una pausa deliberada) y reanudada en una nueva sesión, el agente debe recuperar exactamente el estado en que se encontraba. Esto requiere checkpointing explícito: serializar el estado completo en puntos definidos del proceso, de forma que la recuperación sea posible sin reejecutar los pasos anteriores. Frameworks como LangGraph implementan checkpointing nativo para grafos de agentes, permitiendo reanudar exactamente desde el último checkpoint. El segundo problema es la consistencia en sistemas multiusuario: si el mismo agente gestiona tareas para múltiples usuarios simultáneamente, el aislamiento del estado entre usuarios es un requisito de seguridad crítico.

Nota del Arquitecto: El estado de un agente es un activo que debe gobernarse como cualquier otro dato de negocio. He visto sistemas de agentes que acumulan memoria indefinidamente sin políticas de retención, produciendo perfiles de usuario que contienen años de conversaciones. Aparte del problema de privacidad, esto degrada la calidad del contexto: el agente recibe tanta información sobre el usuario que sus respuestas pierden foco. Una regla práctica es diseñar la memoria del agente con la misma disciplina con que se diseña una base de datos: esquema explícito, política de retención, control de acceso y proceso de purga.

La correcta gestión del estado es lo que convierte a un agente en un sistema de producción robusto. Sin ella, cada sesión comienza desde cero, desperdiciando el contexto acumulado y forzando al usuario a repetir información. Con ella, el agente puede operar como un colaborador que recuerda, aprende y mejora con el tiempo. La sección siguiente explora los patrones de diseño que estructuran el razonamiento y la ejecución del agente.

Módulo 4 – Capítulo 04 – Sección 04

Patrones de Diseño de Agentes

Los patrones de diseño de agentes son las soluciones arquitectónicas reutilizables para los problemas recurrentes que aparecen cuando un modelo de lenguaje debe razonar, ejecutar y evaluar su propio trabajo. A diferencia de los patrones de context engineering del Módulo 3, que describen cómo instruir al modelo, los patrones de diseño de agentes describen cómo organizar el flujo de ejecución del sistema completo. Elegir el patrón incorrecto para el tipo de tarea puede producir agentes que son costosos, impredecibles, o que entran en bucles de los que no pueden salir.

ReAct (Reasoning and Acting) es el patrón más extendido y el punto de partida natural para la mayoría de los agentes. En ReAct, el agente alterna explícitamente entre pasos de razonamiento (Thought) y pasos de acción (Action), seguidos de la observación del resultado (Observation). Este ciclo se repite hasta que el agente considera el objetivo alcanzado. La fortaleza principal de ReAct es la transparencia: el historial de pasos muestra exactamente cómo el agente llegó a su respuesta, lo que facilita la depuración y el análisis de fallos. Su debilidad es la tendencia a bucles: el agente puede entrar en un ciclo donde repite las mismas acciones porque su razonamiento no avanza. Limitadores de iteraciones estrictos y detección de ciclos son controles indispensables. ReAct es la elección correcta para tareas exploratorias donde el camino hacia el objetivo no puede determinarse de antemano.

Planner-Executor separa el razonamiento en dos fases distintas: una fase de planificación (donde el agente genera un plan explícito de pasos antes de ejecutar ninguno) y una fase de ejecución (donde cada paso del plan se ejecuta en secuencia). Esta separación tiene ventajas significativas: el plan puede ser revisado por un humano o por un sistema de validación antes de la ejecución, los pasos del plan pueden ejecutarse en paralelo cuando son independientes, y cuando un paso falla, el agente puede regenerar solo la porción fallida del plan sin reconstruir todo desde el principio. Su limitación es la rigidez ante situaciones imprevistas: si la realidad durante la ejecución difiere significativamente de las suposiciones hechas durante la planificación, el plan puede volverse inválido. Planner-Executor es la elección correcta para

tareas estructuradas con pasos bien definidos donde la predictibilidad y la auditabilidad son más importantes que la flexibilidad.

Supervisor-Workers es un patrón jerárquico donde un agente supervisor descompone la tarea en subtarear y las delega a agentes workers especializados. El supervisor monitorea el progreso de cada worker, integra sus resultados y decide si la tarea está completa o si se requiere trabajo adicional. Este patrón escala naturalmente a tareas complejas: cada worker puede especializarse en un dominio (búsqueda web, análisis de código, generación de documentos), y múltiples workers pueden ejecutarse en paralelo para reducir el tiempo total de la tarea. Los mecanismos de delegación, el monitoreo de sub-agentes y la gestión de timeouts y fallos parciales (¿qué hace el supervisor si un worker no responde?) deben diseñarse explícitamente. Supervisor-Workers es la elección correcta cuando la tarea tiene componentes claramente separables que pueden paralelizarse y cuando se dispone de modelos o herramientas especializadas para cada componente.

Reflection es un patrón donde el agente evalúa su propio output antes de entregarlo. Después de generar una respuesta o completar un paso, el agente aplica un segundo razonamiento crítico: ¿la respuesta cumple con los criterios de calidad definidos? ¿contiene errores factuales detectables? ¿se ajusta al formato requerido? Si la autoevaluación detecta problemas, el agente genera una versión mejorada. Este patrón puede implementarse con el mismo modelo (el LLM evalúa su propio output en una segunda llamada con un prompt diferente) o con un modelo separado dedicado a la evaluación. Reflection es especialmente valioso en sistemas donde la calidad del output es crítica y el costo de un error es alto: generación de código que será ejecutado, redacción de documentos legales o médicos, o respuestas que serán publicadas directamente.

La elección del patrón correcto depende de tres factores:

- **Estructura de la tarea:** ¿el camino hacia el objetivo es predecible (Planner-Executor) o exploratorio (ReAct)? ¿la tarea tiene componentes independientes (Supervisor-Workers)?
- **Costo y latencia:** ReAct puede requerir muchas iteraciones; Planner-Executor permite paralelismo; Supervisor-Workers distribuye el trabajo pero añade overhead de coordinación; Reflection duplica el costo de generación pero reduce el costo de errores.
- **Requisitos de auditabilidad:** si cada paso debe ser explicable y revisable, Planner-Executor y Reflection producen trazas más legibles que ReAct en tareas largas.

Los frameworks de agentes modernos como LangGraph, LlamaIndex Workflows y AutoGen facilitan la implementación de estos patrones con primitivas de alto nivel: grafos de ejecución,

checkpointing, gestión del estado y observabilidad integrada. El conocimiento del patrón correcto para cada caso de uso es la competencia del arquitecto; la elección del framework es una decisión de implementación.

La siguiente sección aplica estos patrones a casos de uso empresariales concretos, donde la elección del patrón y la configuración de sus parámetros produce diferencias medibles en calidad, costo y confiabilidad.

Módulo 4 – Capítulo 04 – Sección 05

Casos de Uso Empresariales

Las arquitecturas de agentes producen valor real cuando se aplican a problemas que comparten características específicas: tareas que requieren múltiples pasos de razonamiento con herramientas, donde el camino exacto no puede determinarse de antemano, y donde el costo de la automatización parcial es sustancialmente menor que el costo de la intervención humana en cada paso. Fuera de esas características, un agente añade complejidad sin beneficio neto. La evaluación honesta de cuándo un agente es la solución correcta es una de las decisiones más valiosas que puede tomar un AI Architect.

Automatización de procesos de análisis e investigación es el caso de uso donde los agentes producen el retorno más claro. Un agente de investigación de mercado puede recibir el nombre de un competidor, buscar información actualizada en la web, analizar sus publicaciones recientes, comparar precios y características con el catálogo propio, y producir un informe estructurado. Este proceso, que un analista humano realizaría en varias horas, puede automatizarse en minutos con un agente que combina búsqueda web, capacidad de lectura de documentos y síntesis estructurada. El patrón Planner-Executor es frecuentemente la elección correcta para este caso, dado que los pasos son conocidos de antemano y pueden paralelizarse.

Soporte y resolución de tickets técnicos es un caso de uso donde los agentes pueden reducir significativamente el volumen de tickets que requieren intervención humana. Un agente de soporte puede recibir una descripción de problema, consultar la base de conocimiento interna (mediante RAG), verificar el estado de sistemas relacionados mediante herramientas de monitoreo, ejecutar scripts de diagnóstico estándar, y resolver el ticket o escalarlo con información de contexto enriquecida. El patrón ReAct es adecuado aquí porque el diagnóstico es exploratorio: el agente no sabe de antemano qué pasos seguirá hasta examinar los síntomas. Los límites de autonomía son críticos en este caso: el agente debe poder diagnosticar sin restricciones, pero solo ejecutar acciones de remediación dentro de un conjunto aprobado.

Asistencia en el desarrollo de software ha producido algunos de los casos de uso de agentes más visibles del mercado. Herramientas como GitHub Copilot Workspace, Devin y Claude Code son implementaciones de agentes que pueden recibir una descripción de tarea, explorar el código existente, planificar los cambios necesarios, implementarlos, ejecutar las pruebas y corregir los fallos detectados. El patrón Reflection es especialmente valioso aquí: el agente evalúa el código generado antes de entregarlo, ejecutando el linter, las pruebas unitarias y verificando el cumplimiento de convenciones. La capacidad de ejecutar código en un sandbox controlado es la herramienta más poderosa — y la más peligrosa si no está correctamente aislada.

Procesamiento documental empresarial es un caso de uso de alto volumen y alto impacto. Un agente de procesamiento de contratos puede recibir un contrato en PDF, extraer las cláusulas relevantes, compararlas con plantillas estándar, identificar desviaciones, clasificarlas por riesgo, y generar un resumen para revisión del equipo legal. Lo que requeriría horas de lectura humana puede realizarse en minutos. Los agentes de procesamiento documental son frecuentemente más simples que otros casos de uso: el objetivo está bien definido, los pasos son predecibles (Planner-Executor), y el output es un documento estructurado que un humano siempre revisa antes de usar.

Gestión de workflows complejos de negocio es el caso de uso más ambicioso y el que requiere mayor madurez arquitectónica. Un agente de onboarding de clientes puede coordinar la solicitud de documentos, validar la información contra bases de datos externas, crear cuentas en múltiples sistemas, enviar comunicaciones personalizadas y monitorear el progreso hasta la activación. Este caso requiere el patrón Supervisor-Workers (múltiples sistemas involucrados), gestión robusta de estado (el proceso puede durar días), integración con sistemas de terceros, y mecanismos claros de intervención humana cuando el agente encuentra situaciones fuera de su competencia.

Los casos donde los agentes no son la solución correcta son igualmente importantes. Si la tarea tiene un proceso perfectamente determinístico, un workflow tradicional (Airflow, Prefect, Luigi) es más confiable, predecible y económico que un agente. Si la tarea puede resolverse con una sola llamada al LLM (una consulta de RAG, una transformación de texto), añadir un agente solo añade latencia y costo. Si el dominio es de altísimo riesgo (cirugía, transacciones financieras a gran escala, sistemas de control críticos), la autonomía de un agente puede ser inaceptable independientemente de su calidad.

La evaluación del caso de uso correcto para un agente debe realizarse antes de cualquier decisión técnica sobre herramientas, modelos o frameworks. El arquitecto que responde

"usaremos un agente" antes de entender las características del problema está repitiendo el error de elegir tecnología antes de entender el problema — el primero de los errores frecuentes descritos en el Capítulo 01.

Módulo 4 – Capítulo 04 – Sección 06

Resumen

Este capítulo desarrolló las arquitecturas de agentes como sistemas con diseño interno propio: componentes específicos con responsabilidades bien definidas, patrones de ejecución que estructuran el razonamiento, y criterios claros de aplicabilidad que distinguen los casos de uso donde los agentes producen valor de los casos donde añaden complejidad innecesaria.

Un agente productivo no es un LLM con acceso a herramientas. Es un sistema compuesto por un modelo de razonamiento, una arquitectura de memoria en dos niveles (episódica y semántica), un registro de herramientas con descripciones precisas, y un bucle de observación-acción con controles explícitos de timeout, límite de iteraciones y política de fallos. Cada uno de esos componentes requiere decisiones de diseño específicas que determinan la calidad, el costo y la seguridad del sistema resultante.

La gestión del estado es la dimensión más frecuentemente subestimada en los proyectos de agentes. Un agente sin gestión robusta de estado es impredecible en tareas que se extienden en el tiempo, incapaz de reanudar trabajo interrumpido, y potencialmente peligroso si mezcla contexto de diferentes usuarios o sesiones. El diseño explícito de las tres capas de estado — sesión, entidad y memoria semántica — con sus políticas de persistencia, retención y control de acceso, es un requisito de arquitectura, no un detalle de implementación.

Los patrones de diseño — ReAct para tareas exploratorias, Planner-Executor para tareas estructuradas paralelizables, Supervisor-Workers para tareas con componentes independientes, Reflection para tareas con alta exigencia de calidad — no son mutuamente excluyentes. Los sistemas de producción frecuentemente combinan patrones: un supervisor que planifica y delega a workers que usan ReAct para sus subtareas, o un ejecutor que aplica reflection sobre el output de cada paso antes de continuar.

Los casos de uso más productivos para agentes empresariales — investigación y análisis, soporte técnico, asistencia al desarrollo, procesamiento documental, gestión de workflows complejos — comparten una característica: el valor de la automatización supera con claridad

el costo y la complejidad del agente. Donde esa condición no se cumple, un pipeline determinístico o un sistema RAG simple es la elección correcta.

El Capítulo 05 extiende estos conceptos al siguiente nivel de complejidad: los sistemas donde múltiples agentes colaboran entre sí. La frontera entre el agente individual de este capítulo y los sistemas multiagente del siguiente es precisa: un agente individual coordina herramientas. Un sistema multiagente coordina agentes. Esa distinción determina los patrones de coordinación, los protocolos de comunicación y los riesgos de gobierno que exploraremos a continuación.

"El verdadero valor de los agentes no reside únicamente en el modelo de lenguaje, sino en la capacidad de integrar razonamiento, herramientas y procesos para resolver problemas reales."

— Principio de diseño de sistemas agénticos